

Dokumentacja Błędów i Problemów Technicznych

Podczas realizacji projektu **Terminal Arcade** napotkano kilka kluczowych problemów technicznych związanych ze specyfiką programowania gier konsolowych w standardzie C++. Poniżej przedstawiono kompletny rejestr 9 najważniejszych błędów oraz metody ich usunięcia.

Błąd 1: Rozpadające się tabele i "krzaczk" zamiast symboli (Błąd kodowania znaków)

Gdzie miało to miejsce: Interfejs wizualny w grach 2048 oraz Blackjack (rozwiązano w `main.cpp`).

Kiedy wystąpiło: Po zaimplementowaniu prawdziwych symboli karcianych (♥, ♦) w Blackjaku oraz podczas testowania gry 2048 w środowisku Windows.

Skąd się wzięło: Konsola systemowa Windows (Command Prompt) domyślnie wykorzystuje przestarzałe kodowanie tekstu (OEM/ANSI). Próba wyrenderowania wielobajtowych znaków Unicode (UTF-8) używanych do rysowania statków, kart czy klocek kończyła się niepowodzeniem.

Co robił błąd: Zamiast symboli pojawiały się ciągi losowych znaków (np. `[ 8 000 ]`). Co gorsza, terminal odczytywał jeden znak wielobajtowy jako kilka oddzielnych znaków, co całkowicie psuło matematykę odpowiedzialną za liczenie długości stringów i wyrównywanie tabel (spowodowało to rozsypanie się siatki w 2048).

Jak naprawiono: Zmodyfikowano główny plik `main.cpp`. Przy użyciu dyrektyw preprocesora `#if defined(_WIN32)` załączono bibliotekę `<windows.h>` i na samym starcie programu wymuszono zmianę strony kodowej konsoli poprzez wywołanie instrukcji z Windows API: `SetConsoleOutputCP(CP_UTF8)` oraz `SetConsoleCP(CP_UTF8)`.

Błąd 2: Uciążliwe migotanie ekranu (Terminal Flickering)

Gdzie miało to miejsce: Moduł renderujący obraz (`Display.cpp` i `Display.h`), co krytycznie wpływało na gry `Snake.cpp` , `Bombberman.cpp` i `CyberDefense.cpp` .

Kiedy wystąpiło: W momencie przejścia z gier turowych na gry akcji działające w czasie rzeczywistym, które wymagały odświeżania na poziomie około 20 klatek na sekundę (FPS).

Skąd się wzięło: Standardowe podejście do rysowania polegało na użyciu funkcji `Display::clearScreen()` , która mechanicznie czyściła cały bufor terminala przed wyrenderowaniem nowej klatki.

Co robił błąd: Czyszczenie ekranu z tak dużą częstotliwością powodowało, że ludzkie oko rejestrowało ułamki sekund, w których ekran był całkowicie czarny, co objawiało się silnym, drażniącym migotaniem (stroboskopem).

Jak naprawiono: Opracowano i zaimplementowano funkcję `Display::clearScreen()` opartą na dwukrotnym buforowaniu. Zamiast czyścić ekran, strumień `std::cout` jest przekierowywany do `std::stringstream` , a po skończeniu rysowania klatki, kursor powraca na początek ekranu za pomocą ANSI Escape Code `\033[H` . Stara klatka jest nadpisywana przez nową, co całkowicie wyeliminowało migotanie.

Błąd 3: Zatrzymywanie pętli gry (Blokujące wejście)

Gdzie miało to miejsce: Moduł obsługi klawiatury (`Input.h`) oraz główne pętle wszystkich gier czasu rzeczywistego.

Kiedy wystąpiło: Podczas pierwszych prób uruchomienia gry Snake, gdzie wąż musiał poruszać się samoistnie.

Skąd się wzięło: Klasyczne metody pobierania wejścia w C++ (takie jak `std::cin` czy `getchar()`) mają charakter synchroniczny (blokujący) – wstrzymują one całkowicie wątek programu do momentu, aż użytkownik wciśnie klawisz.

Co robił błąd: Powodował, że gry czasu rzeczywistego działały jak gry turowe. Elementy na ekranie (wąż, wrogowie, odliczanie bomb) zamrażały się i nie aktualizowały swoich pozycji, dopóki gracz nie wykonał ruchu.

Jak naprawiono: Moduł `Input.h` został przepisany w celu wdrożenia asynchronicznego (nieblokującego) sprawdzania stanu klawiatury za pomocą funkcji `hasInput()` . Na platformie Windows wykorzystano bibliotekę `<conio.h>` i funkcję `_kbhit()` , natomiast na systemach Unix użyto `<termios.h>` modyfikując deskryptor pliku za pomocą `fcntl` i flagi `O_NONBLOCK` . Dzięki temu pętla gry może weryfikować wciśnięcie klawisza "w locie" nie przerywając działania silnika.

Błąd 4: Desynchronizacja bufora wejściowego oraz uszkodzenie bazy CSV (Brak walidacji nazwy gracza)

Gdzie miało to miejsce: Zapis najlepszych wyników do pliku we wszystkich grach obsługujących tablicę wyników (`Snake.cpp` , `Game2048.cpp` , `MineSweeper.cpp` , `Blackjack.cpp`).

Kiedy wystąpiło: Podczas wprowadzania nowej nazwy gracza na ekrany post-game w przypadku uzyskania najlepszego wyniku (High Score).

```
std::cin >>
playerName Jan Kowalski std::cin pobierał
```

tylko pierwsze słowo, a drugie zostawało w buforze strumienia wejściowego, co psuło działanie sterowania w kolejnych pętlach menu. Dodatkowo wpisanie przecinka w nazwie uszkadzało parser bazy CSV (`scores.txt`), ponieważ przecinek jest znakiem podziału pól.

Co robił błąd: Powodował rozsynchronizowanie odczytu klawiatury w menu głównym (program odczytywał pozostałe w buforze litery jako intencje gracza) oraz łamał format zapisu tabeli rekordów (baza danych zapisywała nieprawidłową liczbę kolumn w CSV).

Jak naprawiono: Zaimplementowano bezpieczną funkcję `getValidPlayerName()` w `Input.h` . Pobiera ona całą linię tekstu (`std::getline`), odrzuca puste nazwy, usuwa białe znaki skrajne oraz odrzuca próby zapisu nazw zawierających spacje wewnętrzne lub przecinki, żądając od użytkownika wprowadzenia poprawnego ciągu.

Błąd 5: Ucinanie znaków wielobajtowych (Uszkodzenie pamięci UTF-8)

Gdzie miało to miejsce: Metoda `drawTable()` w grze `Blackjack.cpp` .

Kiedy wystąpiło: Podczas zamiany tekstowych nazw kolorów kart (np. "Hearts") na prawdziwe symbole Unicode (♥, ♦, ♣, ♠).

Skąd się wzięło: Pierwotny kod używał metody `substr(0,1)` na obiekcie `std::string` , aby wyciągnąć pierwszą literę koloru (np. 'H' z "Hearts"). Znaki ASCII zajmują 1 bajt, więc to działało. Znaki Unicode (np. ♥) zajmują w C++ wiele bajtów.

Co robił błąd: Wywołanie `substr(0,1)` na symbolu UTF-8 dosłownie przecinało znak w połowie jego bajtów. Próba wypisania na ekran połowy znaku powodowała wyświetlanie "śmieci" w pamięci konsoli, co psuło cały interfejs stołu do Blackjaka.

Jak naprawiono: Całkowicie usunięto wywołanie `substr(0,1)` przy formatowaniu kart. Zamiast tego zadeklarowano w strukturze pełne, wielobajtowe znaki jako stringi i wypisywano je w całości, polegając na wcześniejszym wymuszeniu trybu UTF-8 w konsoli.

Błąd 6: Naruszenie pamięci przy rotacji (Array Out of Bounds)

Gdzie miało to miejsce: Tryb manualnego rozstawiania statków w `Battleship.cpp` (`manualPlaceShips()`).

Kiedy wystąpiło: Podczas implementacji obracania statków (klawisz 'R') przy krawędziach planszy (10x10).

Skąd się wzięło: Tablica `playerBoard` miała sztywny rozmiar. Cursor ("duch" statku) mógł znajdować się np. na koordynacie X=8. Jeśli gracz miał w ręku lotniskowiec (długość 5) i obrócił go z pionu do poziomu, jego współrzędne nagle zajmowały pola od 8 do 12.

Co robił błąd: Odwołanie się do wektora `playerBoard[y][x]` gdzie `x = 12` próbowało zapisać dane w obszarze pamięci nienależącym do planszy, co natychmiast wywoływało krytyczny błąd i zamknięcie programu (Segmentation Fault).

Jak naprawiono: Dodano w locie matematyczne klamrowanie (clamping) w instrukcji sterującej klawiszem 'R'. Po obróceniu statku silnik sprawdza, czy `cx + len > 10`. Jeśli tak, automatycznie przesuwa cursor statku z powrotem na planszę (`cx = 10 - len`), zapobiegając wyjściu poza indeks tablicy.

Błąd 7: Nieskończone klonowanie interfejsu (Terminal Menu Stacking)

Gdzie miało to miejsce: Ekran końcowy (Post-Game Menu) w grach takich jak Tic-Tac-Toe, Saper czy 2048.

Kiedy wystąpiło: Podczas przejścia ze standardowych menu wpisywanych z klawiatury (1, 2, Q) na interaktywne menu nawigowane klawiszami WASD.

Skąd się wzięło: Pętla `while(!madeChoice)` oczekująca na ruch strzałkami nie miała odpowiedniego mechanizmu odświeżania obszaru rysowania.

Co robił błąd: Z każdym wciśnięciem klawisza W lub S, aby przesunąć kursor, stara linijka tekstu pozostawała na ekranie, a nowe, zaktualizowane menu drukowało się poniżej. Skutkowało to tworzeniem się ciągnącej się w nieskończoność "rolki" z menu na dole terminala, psując cały efekt Arcade.

Jak naprawiono: Dodano system bezwarunkowego odświeżania pod koniec pętli wyboru. Zastosowano `Display::clearScreen()` (lub wywołanie przerysowania całej planszy gry za menu) natychmiast po każdej zmianie indeksu `endSel`, co utrzymuje menu precyzyjnie w jednym, statycznym miejscu na ekranie.

Błąd 8: Wyczerpanie stosu w grze (Deck Exhaustion / Infinite Loop)

Gdzie miało to miejsce: Generowanie i dociąganie kart w `Blackjack.cpp` (`drawCard()`).

Kiedy wystąpiło: Podczas projektowania długich sesji gry, gdzie gracz nie wychodzi do menu głównego, tylko gra partię za partią.

Skąd się wzięło: Wektor `deck` nie odnawiał się automatycznie.

Co robił błąd: Kiedy wektor kart opustoszał po kilkunastu rozdaniach, wywołanie `deck.back()` próbowało pobrać element z pustego wektora, a `deck.pop_back()` usunąć nieistniejący obiekt, co prowadziło do natychmiastowego rzucenia wyjątku C++ i zamknięcia programu.

Jak naprawiono: Zaimplementowano klasyczne "kasynowe" rozwiązanie (Shoe). Generowane są od razu 4 pełne talie naraz (4 w pętli zewnętrznej * 52 karty), a w funkcji `drawCard()` dodano bezpiecznik `if (deck.empty()) buildAndShuffleDeck();`, który w ułamku sekundy i całkowicie niewidocznie dla gracza, tworzy nową pulę kart.

Błąd 9: Pętla nieskończona i zawieszenie gry przy wczytywaniu danych liczbowych (Fail State strumienia `std::cin`)

Gdzie miało to miejsce: Tryb konfiguracji niestandardowej planszy (`Custom`) w grze `MineSweeper.cpp`.

Kiedy wystąpiło: W momencie wprowadzania przez użytkownika własnych parametrów planszy: szerokości (`Width`), wysokości (`Height`) oraz liczby min (`Mines`).

Skąd się wzięło: Wczytywanie zmiennych liczbowych odbywało się bezpośrednio przez `std::cin >> width` bez weryfikacji stanu strumienia.

Co robił błąd: Wpisanie przez użytkownika ciągu znaków tekstowych (np. "abc") zamiast cyfry wprowadzało strumień wejściowy konsoli w trwały stan błędu (`fail state`).

Program ignorował wszystkie kolejne próby pobrania danych, przypisywał zmiennym wartości śmieciowe i wchodził w nieskończoną pętlę renderowania ekranu z błędnymi wymiarami, co całkowicie zamrażało aplikację.

```
getValidInteger(min, max, prompt) Input.h
```

bezpiecznego rzutowania `std::stoi`, wyłapuje potencjalne wyjątki (`invalid_argument`, `out_of_range`) oraz automatycznie resetuje flagi błędów strumienia wejściowego (`std::cin.clear()`, `std::cin.ignore()`), jeśli podana wartość była niepoprawna.